30

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Zarif Kapatma (Graceful Shutdown): shutdown, shutdownNow & Zehirli Hap (Poison Pill)

Graceful Shutdown Patterns: shutdown(), shutdownNow() & The Poison Pill Pattern

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              THREAD POOL ZARİF KAPATMA (GRACEFUL SHUTDOWN)   │
└──────────────────────────────────────────────────────────────┘

ADIM 1: executor.shutdown();
  ├──► Yeni gelen görevleri reddeder! (Rejection)
  └──► Kuyrukta bekleyen mevcut görevleri çalıştırmaya devam eder.

ADIM 2: executor.awaitTermination(30, TimeUnit.SECONDS);
  ├──► 30 saniye boyunca mevcut işlerin bitmesini kibarca bekle.

ADIM 3: Eğer 30 saniyede bitmediyse? ZORLA KAPAT:
  ├──► executor.shutdownNow(); ──► Çalışan thread'lere interrupt() çeker!
  └──► Kuyrukta hiç başlamamış görevlerin listesini döner (List<Runnable>).

ADIM 4: executor.awaitTermination(10, TimeUnit.SECONDS);
  └──► Kapanışı kesin olarak doğrula.
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranın kapanış saati gibidir: 1. Adım (`shutdown()`): Kapı kilitlenir, yeni müşteri alınmaz. Ama masada yemeğini yiyen müşteriler kovulmaz; yemeklerini bitirmeleri beklenir. 2. Adım (`awaitTermination()`): Garsonlar masaların kalkmasını 30 dakika bekler. 3. Adım (`shutdownNow()`): 30 dakika geçmesine rağmen kalkmayan inatçı müşteri olursa ışıklar söndürülür, müzik kapatılır ve güvenlik herkesi dışarı çıkarır (Thread Interrupt)! İşte zarif kapatma, veri kaybı yaşamadan sistemi güvenle kapatma sanatıdır!

🇬🇧 ENGLISH (ELI5)

Shutting down a thread pool is like closing a restaurant: Step 1 (`shutdown()`): Lock the front door to new customers. Do not evict seated patrons eating dinner; let them finish. Step 2 (`awaitTermination()`): Wait patiently for 30 minutes for tables to clear. Step 3 (`shutdownNow()`): If stubborn diners refuse to leave after 30 minutes, cut the power, turn off the music, and usher them out via security (`Thread.interrupt()`). Graceful shutdown guarantees zero data loss.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üretim Seviyesinde 2-Aşamalı Kapatma Deseni (Two-Phase Termination): 1) `shutdown()` vs `shutdownNow()`: `shutdown()` havuzun durumunu SHUTDOWN yapar; yeni iş almaz ama kuyruktaki işleri tamamlar. `shutdownNow()` havuzu STOP durumuna çeker, kuyruktaki bekleyen görevleri çalıştırmadan bir `List\<Runnable>` olarak geri döner ve çalışan işçilere `interrupt()` sinyali gönderir. 2) Poison Pill Deseni: Kuyruk tabanlı Producer-Consumer sistemlerinde özel bir 'Zehirli Nesne' (ör. `POISON_PILL = new Task(-1)`) kuyruğa atılır. Tüketici bu nesneyi gördüğü anda döngüden çıkarak kendini sonlandırır; böylece `interrupt()` zorlamasına gerek kalmadan tamamen zarif bir kapanış sağlanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Oracle Standard 2-Phase Termination Protocol: 1) State Transitions: `shutdown()` transitions state from RUNNING to SHUTDOWN, refusing incoming tasks while executing buffered queue items. `shutdownNow()` transitions state directly to STOP, actively calling `Thread.interrupt()` on workers and draining unstarted tasks into a `List\<Runnable>` return payload. 2) The Poison Pill Architecture: In high-throughput messaging pipelines, a designated Sentinel Object (the Poison Pill) is injected into the queue. Consumers break their processing loops upon encountering the pill, achieving deterministic, exception-free termination without relying on thread interrupts.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Resilient Two-Phase Graceful Shutdown Deseni
import java.util.concurrent.*;

public class GracefulShutdownManager {
    public static void shutdownGracefully(ExecutorService pool, long timeoutSeconds) {
        pool.shutdown(); // 1. Yeni iş kabulünü durdur
        try {
            // 2. Mevcut işlerin bitmesini bekle
            if (!pool.awaitTermination(timeoutSeconds, TimeUnit.SECONDS)) {
                IO.println("⚠ İşler zamanında bitmedi, zorla kapatılıyor (shutdownNow)...");
                var droppedTasks = pool.shutdownNow(); // Çalışanlara interrupt çek
                IO.println("İptal edilen kuyruk görev sayısı: " + droppedTasks.size());

                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
                    IO.println("❌ Havuz tamamen kapatılamadı!");
                }
            } else {
                IO.println("✔ Havuz tüm görevleri başarıyla tamamlayıp kapandı.");
            }
        } catch (InterruptedException e) {
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }

    void main() {
        var pool = Executors.newFixedThreadPool(2);
        pool.execute(() -> {
            try { Thread.sleep(500); } catch (InterruptedException ignored) {}
        });
        shutdownGracefully(pool, 2);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'shutdownNow() çağrıldığında çalışan thread anında pat diye durur mu?' Yanıt: 'HAYIR! Java'da thread'leri dışarıdan zorla öldüren bir mekanizma yoktur (Thread.stop() deprecated ve tehlikelidir). shutdownNow() yalnızca thread'e interrupt() sinyali gönderir; eğer çalışan kod InterruptedException yakalamıyor veya Thread.interrupted() kontrol etmiyorsa çalışmaya devam eder!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Java Truth: 'Does shutdownNow() kill running threads instantly?' Senior Answer: 'NO! Java has no safe pre-emptive thread-killing primitive (`Thread.stop()` is deprecated and dangerous). `shutdownNow()` merely issues a cooperative `interrupt()` signal. If the executing code does not poll `Thread.interrupted()` or sleep, it runs forever!'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- shutdown() kuyruktaki işleri bitirir; shutdownNow() çalışanlara interrupt çeker.
- Standart 2-aşamalı kapatma deseni: shutdown() -> awaitTermination() -> shutdownNow().
- Java thread'leri zorla öldürülemez; cooperative interruption mantığına uymak zorundadır.

🎯 Key Takeaways for the Interview

- shutdown() drains the queue; shutdownNow() interrupts active workers and returns unexecuted tasks.
- Always apply the 2-phase termination protocol: shutdown() -> await -> shutdownNow().
- Threads cannot be forcefully killed; they must cooperatively handle interruption.